

EPGP Academic Scheduling Optimization using Linear Programming

Devadatta Mahesh Pokharanakar
Department of Data Science, IIT Palakkad

June 6, 2026

Introduction

This formulation represents the scheduling problem for the IIM EPGP Program, accommodating different sections/areas, courses, time slots, and faculty members. The objective is to create an optimal schedule that satisfies all hard constraints while minimizing violations of a configurable consecutive-period soft constraint.

1 Sets and Indices

- $b \in B$: Set of buckets. Buckets are either section identifiers (in Sections mode) or area short-names (in Areas mode).
- $c \in C$: Set of Courses.
- $t \in T_b$: Set of time slots available for bucket b .
- $f \in F$: Set of Faculty members.
- $d \in D$: Set of unique Calendar Dates in the quarter.
- $pk \in PK_b$: Set of **period keys** for bucket b — unique integers that identify each scheduling period. The encoding is determined by the configured period unit:

- `period_unit = "weeks"`: $pk = \text{iso_year} \times 100 + \text{iso_week}$ (e.g. Week 10 of 2026 $\rightarrow$ 202610)
- `period_unit = "days"`: $pk = \text{date.toordinal}()$ (unique integer per calendar day)

Two period keys pk and pk' are **consecutive** if they represent immediately adjacent periods — successive ISO weeks (with correct year-rollover handling) or successive calendar days. Note that *arithmetic* $pk + 1$ is **not** the same as the consecutive successor for week-encoded keys (e.g. 202652 $\rightarrow$ 202701, not 202653). We write $\text{succ}^j(pk_0)$ for the j -th consecutive successor of pk_0 .

- $g \in G$: Set of course conflict groups (used by optional constraint $H7$).
- $C_g \subseteq C$: Set of courses in conflict group g .
- $B_g \subseteq B$: Set of buckets to which conflict group g applies.

2 Parameters

- $R_{b,c}$: Required number of sessions for bucket b in course c .
- $\text{Faculty}(b, c)$: The faculty member assigned to teach course c in bucket b .
- $\text{Date}(b, t)$: The calendar date associated with slot t for bucket b .
- $\text{Period}(b, t)$: The period key pk associated with slot t for bucket b (see PK_b above).

- **Boundary(b, t):** The boundary-group key of slot t for bucket b . Controls where the consecutive counter resets:
 - `reset_boundary = "month"`: $\text{Boundary}(s, t) = \text{year} \times 100 + \text{month}$
 - `reset_boundary = "none"`: $\text{Boundary}(s, t) = 0$ (no reset; sequence is continuous)
- **Unavailable(f, d):** Binary flag; 1 if faculty f is unavailable on date d , 0 otherwise. (*used by optional constraint H6*)
- M_f : Maximum sessions faculty f may teach per calendar day.
- M : Maximum number of **consecutive periods** in which the same course may be taught to the same scope without incurring a penalty (user-configured; default $M = 2$). A window of $M + 1$ consecutive same-boundary periods that all have active sessions constitutes one violation.
- **Scope:** The consecutive-rule and auxiliary variables operate over a configurable *scope*:
 - Sections mode: $\text{scope} = (b, c)$ (bucket-course pair, i.e., per-section behavior)
 - Areas mode: $\text{scope} = c$ (course-level scope aggregated across buckets/areas)

3 Decision Variables

- $x_{b,c,t} \in \{0, 1\}$: 1 if bucket b is assigned course c at time slot t ; 0 otherwise. (In Sections mode this enforces one course per (b, t) ; in Areas mode multiple concurrent $x_{b,c,t} = 1$ for the same (b, t) are allowed unless forbidden by other constraints.)
- $y_{\text{scope}, pk} \in \{0, 1\}$: Auxiliary variable; 1 if the given *scope* has at least one session of the relevant course in period pk ; 0 otherwise.
- $p_{\text{scope}, pk_0} \in \{0, 1\}$: Penalty variable; 1 if the scope is active in *every* period of the window $(pk_0, \text{succ}(pk_0), \dots, \text{succ}^M(pk_0))$; 0 otherwise. Created only when the consecutive rule is enabled.

4 Objective Function

The objective depends on whether the consecutive sessions rule is enabled in the configuration.

Case 1 — Consecutive rule enabled (Soft):

$$\text{Minimize } Z = \sum_{s \in S} \sum_{c \in C} \sum_{pk_0} p_{s,c,pk_0} \quad (1)$$

Minimize the total number of windows in which the same course is taught in $M + 1$ or more consecutive same-boundary periods for the same section.

Case 2 — Consecutive rule disabled (Pure feasibility):

$$\text{Minimize } Z = 0 \quad (2)$$

No soft objective; the solver simply finds any schedule satisfying all active hard constraints.

5 Constraints

Fixed Hard Constraints (always applied)

- **H1: Session Requirement**

$$\sum_{t \in T_b} x_{b,c,t} = R_{b,c} \quad \forall b \in B, c \in C \quad (3)$$

Every bucket must complete the required number of sessions for every assigned course.

- **H2: At Most One Course per Slot**

$$\sum_{c \in C} x_{b,c,t} \leq 1 \quad \forall b \in B, t \in T_b \quad (\text{Sections mode only}) \quad (4)$$

In Sections mode each available time slot for a bucket (section) may be occupied by at most one course, so unused slots are allowed when total available capacity exceeds required sessions. This constraint is not applied in Areas mode, where areas may host parallel sessions in the same slot.

- **H3: No Faculty Cloning**

$$\sum_{\substack{(b,c): \\ \text{Faculty}(b,c)=f \\ \text{Date}(b,t)=d, \text{FromTime}(b,t)=\tau}} x_{b,c,t} \leq 1 \quad \forall f \in F, (d, \tau) \quad (5)$$

A faculty member cannot teach more than one bucket at the same date and start time.

- **H4: Maximum Daily Workload**

$$\sum_{b \in B} \sum_{c \in C} \sum_{\substack{t \in T_b: \\ \text{Date}(b,t)=d, \\ \text{Faculty}(b,c)=f}} x_{b,c,t} \leq M_f \quad \forall f \in F, d \in D \quad (6)$$

A faculty member may teach at most M_f sessions per calendar date across all buckets.

- **H5: Course Spacing (Daily)**

$$\sum_{\substack{t \in T_b: \\ \text{Date}(b,t)=d}} x_{b,c,t} \leq 1 \quad \forall b \in B, c \in C, d \in D \quad (7)$$

A bucket cannot be scheduled for the same course more than once on the same calendar day.

Optional Hard Constraints (can be disabled via configuration)

- **H6: Faculty Unavailability** (*Optional*)

$$x_{b,c,t} = 0 \quad \text{if } \text{Unavailable}(\text{Faculty}(b,c), \text{Date}(b,t)) = 1 \quad \forall b \in B, c \in C, t \in T_b \quad (8)$$

A class cannot be scheduled on a date when the assigned faculty member is marked unavailable.

- **H7: Course Conflict Groups (Bucket Scope)** (*Optional*)

$$\sum_{b \in B_g} \sum_{c \in C_g} \sum_{\substack{t: \\ \text{Date}(b,t)=d, \\ \text{FromTime}(b,t)=\tau}} x_{b,c,t} \leq 1 \quad \forall g \in G, (d, \tau) \quad (9)$$

For each conflict group g , at most one course from C_g may run at any given date and start time across the applicable buckets B_g .

6 Soft Constraint: Consecutive Sessions Rule

When the consecutive rule is enabled, we penalize every window in which the same course is taught to the same section in $M + 1$ or more consecutive same-boundary periods. The formulation uses auxiliary period-activity indicators y to link slot-level decisions x to the period-level penalty variables p .

Window Eligibility

A window $\mathcal{W} = (pk_0, \text{succ}(pk_0), \dots, \text{succ}^M(pk_0))$ of length $M + 1$ over (s, c) is **eligible** if and only if:

1. **Coverage:** All $M + 1$ period keys in $\mathcal{W}$ exist in PK_s for section s with valid slots mapped to course c .
2. **Consecutiveness:** Each successive pair in $\mathcal{W}$ are adjacent periods (next ISO week, or next calendar day), including correct year-boundary handling.
3. **Boundary confinement** (when `reset_boundary` $\neq$ "none"):

$$\text{Boundary}(s, t_0) = \text{Boundary}(s, t_j) \quad \forall j = 1, \dots, M \quad (10)$$

where t_0, t_j are representative slots in pk_0 and $\text{succ}^j(pk_0)$ respectively. This ensures the consecutive counter resets at month (or other configured) boundaries, so violations cannot span across them.

Auxiliary Constraints (linking y to x)

- **A1: Period Activity — Lower Bound**

$$y_{\text{scope}, pk} \geq x_{b,c,t} \quad \forall b \in B, c \in C, t \in T_b \text{ where } \text{Period}(b, t) = pk \text{ and scope applies} \quad (11)$$

$y_{\text{scope}, pk}$ is forced to 1 if *any* session within the scope is scheduled in period pk .

- **A2: Period Activity — Upper Bound**

$$y_{\text{scope}, pk} \leq \sum_{\substack{t \in T_b \\ \text{Period}(b, t) = pk \\ \text{and scope applies}}} x_{b,c,t} \quad \forall pk \in \bigcup_b PK_b \quad (12)$$

$y_{\text{scope}, pk}$ is forced to 0 if *no* session within the scope is scheduled in period pk .

Penalty Activation

- **P: Consecutive Period Penalty**

For every eligible window $\mathcal{W} = (pk_0, \text{succ}^1(pk_0), \dots, \text{succ}^M(pk_0))$ over the chosen scope:

$$p_{\text{scope}, pk_0} \geq \sum_{pk \in \mathcal{W}} y_{\text{scope}, pk} - M \quad (13)$$

Activation logic:

- If every period in $\mathcal{W}$ is active ($y = 1$ for all $M + 1$ periods): $\sum_{pk \in \mathcal{W}} y_{\text{scope}, pk} = M + 1$, so RHS = 1, forcing $p_{\text{scope}, pk_0} \geq 1$ — the penalty fires.
- If at most M periods are active: $\sum_{pk \in \mathcal{W}} y_{\text{scope}, pk} \leq M$, so RHS ≤ 0 . Since $p \geq 0$ always holds, the constraint is slack and p remains 0.

Special cases recovered by this general formulation:

- $M = 1$, `reset_boundary` = "none": reduces to the classic *Alternate-Week Rule* — the same course cannot be taught in two consecutive weeks within the configured scope.
- $M = 2$, `reset_boundary` = "month": implements the *Two-Consecutive-Week Rule within a Month* — within the chosen scope the course may appear in at most 2 consecutive weeks per month; a third consecutive week in the same month triggers a penalty.

Implementation Notes

The solver implementation generalizes the original per-section model by introducing a bucket abstraction and a configurable *scope* for the consecutive-rule. Key points:

- The decision variables and constraints are constructed over buckets B so the same code supports Sections and Areas modes.
- The consecutive-rule auxiliary variables y and penalties p are indexed by the chosen scope (either (b, c) pairs or course-level c) so penalties are computed correctly in each mode.
- Helper routines compute period adjacency and boundary keys robustly across year boundaries (ISO week rollovers) to ensure correct window eligibility checks.